Lab Handout 2-A

Module 2 Project Orientation, Field-Programmable Gate Array (FPGA) Passthrough, and Capture Set

Capstone — Module 2: FPGA Register-Transfer Level (RTL) Design and Signal Ingestion

1 Objective

Open the `ingest_top` Vivado project built in LH1-H and prepare it for incremental RTL development across Module 2. The five VHSIC Hardware Description Language (VHDL) files you wrote in LH1-H form the bring-up baseline; later handouts replace each block one at a time. By the end of this handout the project will have a simulation fileset, an FPGA bitstream that simultaneously drives the seven-segment display *and* echoes the raw Threat byte stream out the Nexys A7's on-board USB-UART, two raw `.bin` captures copied into a `data/` folder for the Module 2 testbenches to consume, and a documented baseline utilization snapshot.

2 Prerequisites

- Lab Handout 1-H complete: `ingest_top` synthesizes, implements, programs, and produces a live Beacon ID on the seven-segment display.
- Target A, Target B (LH1-F), and the Threat (LH1-G) all flashed and known to be ping-ponging on the radio.
- Host PC with Python 3.9+ and the `pyserial` package installed (`pip install pyserial`).

3 Materials

- Digilent Nexys A7-100T development board.
- Host PC with Vivado ML Standard installed.
- The LH1-H `ingest_top` project directory.
- USB cable connecting the Nexys A7 to the host (the same cable used to program the board).

Module 2 development model

A single `ingest_top` project accumulates all Module 2 RTL. Rather than starting over, each handout (LH2-B through LH2-F) replaces exactly one VHDL file with a new student-written version and adds a corresponding testbench to the simulation fileset. This keeps synthesis, timing, and resource numbers directly comparable across sessions, and the project arrives at Module 3 in a single coherent state. **Do not delete the LH1-H files yet** — each Module 2 handout instructs you exactly when to remove which file from the project.

Why we capture `.bin` files in this handout

The Module 2 testbenches (LH2-B regression and LH2-G end-to-end) replay a recorded byte stream into the Hardware Description Language (HDL) under simulation. That recording has to come from somewhere. Earlier course revisions captured it via a separate USB-to-UART adapter wired to the Threat's `USART3 TX` pin. To remove that hardware dependency, this handout adds one line of VHDL and one line of Xilinx Design Constraints (XDC) that route the same byte stream out the Nexys A7's onboard FT2232 USB-UART. The same USB cable that programs the board also enumerates as a COM port on the host, and a tiny Python script captures the bytes to disk.

4 Procedure

4.1 Step 1 — Open the LH1-H project

1. Launch Vivado. On the start page, click the `ingest_top` entry under **Recent Projects**, or use *File* → *Open Project* and navigate to `ingest_top.xpr`.
2. Confirm the Sources panel shows all five LH1-H files (`uart_rx.vhd`, `byte_fifo.vhd`, `frame_detector.vhd`, `seg7_driver.vhd`, `ingest_top.vhd`) and that `ingest_top` is bold (set as Top).
3. Confirm the bottom-right part indicator reads `xc7a100tcsg324-1`.

4.2 Step 2 — Add a simulation fileset

1. In the Flow Navigator, click **Project Settings** → **Simulation**. Confirm the *Simulation top module name* field is writable — if no `sim_1` fileset exists yet, Vivado may prompt you to create one. Accept.
2. In the Sources panel, select the **Simulation Sources** tab. Confirm the `sim_1` set is present. All testbenches added in LH2-B through LH2-G will go here.

Keeping simulation sources isolated

Adding testbenches only to the simulation fileset (never to Design Sources) prevents them from appearing in the synthesis hierarchy. Vivado will not attempt to synthesize them.

4.3 Step 3 — Create the `data/` folder

1. In the project directory (the folder containing `ingest_top.xpr`), create a `data/` subfolder alongside the auto-generated `ingest_top.srcs/`, `ingest_top.runs/`, `ingest_top.sim/`, and `ingest_top.cache/` folders. Modern Vivado prefixes these working folders with the project name; if your project is named differently you will see the same suffixes prefixed with that name.
2. Leave `data/` empty for now — Step 6 below produces the `.bin` captures that go into it.
3. Note: `data/` is not added to Vivado's Sources panel — it is a plain file-system folder that testbenches open via `std.textio`.

4.4 Step 4 — Add the USB-UART passthrough to the bitstream

You will add one output port to `ingest_top`, route the raw `uart_rx_pin` signal directly to it, and uncomment the XDC entry that maps that port to the FT2232's UART RX pin (FPGA pin `D4`, called `UART_RXD_OUT` in the master XDC). The Threat's `USART3 TX` stream is already at 115200 8N1; the FT2232 channel exposed to the host is also 115200 8N1; both ends agree on framing, so a direct wire echoes the byte stream byte-for-byte without any re-serialization.

Why a direct wire works

A Universal Asynchronous Receiver/Transmitter (UART) "adapter" at the silicon level is just a baud generator and a shift register on each end. As long as both ends are clocked to the same baud rate and agree on word format, the line in between is just digital 1s and 0s. Routing the FPGA's incoming RX pin through to its outgoing TX-to-host pin makes the FPGA look like a passive piece of wire from the host's perspective. The FPGA also runs the `uart_rx` block in parallel, so the same bits drive both the seven-segment display *and* the host capture — the two paths are completely independent.

Step 4.1 — Edit `ingest_top.vhd`

Open `ingest_top.vhd`. Add a new output port `UART_RXD_OUT` to the entity, then add one assignment statement at the bottom of the architecture that drives that port from the existing `uart_rx_pin` input. The complete file should now read:

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ingest_top is
    port (
        CLK100MHZ    : in  std_logic;
        CPU_RESETN   : in  std_logic;                        -- active-low pushbutton
        uart_rx_pin  : in  std_logic;                        -- PMOD JA pin 1 (C17)
        UART_RXD_OUT : out std_logic;                        -- onboard USB-UART, FPGA -> host (D4)
        CA, CB, CC, CD, CE, CF, CG : out std_logic;          -- 7-seg segments (active low)
        DP           : out std_logic;
        AN           : out std_logic_vector(7 downto 0);
        LED          : out std_logic_vector(15 downto 0)
    );
end ingest_top;

architecture rtl of ingest_top is
    signal rst_n      : std_logic;

    signal rx_data    : std_logic_vector(7 downto 0);
    signal rx_strobe  : std_logic;

    signal fifo_data  : std_logic_vector(7 downto 0);
    signal fifo_rd_en : std_logic;
    signal fifo_empty : std_logic;
    signal fifo_full  : std_logic;

    signal beacon_id  : std_logic_vector(7 downto 0);
    signal new_frame  : std_logic;

    signal seg_bus    : std_logic_vector(6 downto 0);
begin
    rst_n <= CPU_RESETN;                                     -- already active-low

    u_uart : entity work.uart_rx
```

```
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  rx => uart_rx_pin,
                  rx_data => rx_data, rx_strobe => rx_strobe);

    u_fifo : entity work.byte_fifo
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  wr_en => rx_strobe, wr_data => rx_data,
                  rd_en => fifo_rd_en, rd_data => fifo_data,
                  empty => fifo_empty, full => fifo_full);

    u_det : entity work.frame_detector
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  fifo_empty => fifo_empty, fifo_data => fifo_data,
                  fifo_rd_en => fifo_rd_en,
                  beacon_id => beacon_id, new_frame => new_frame);

    u_seg : entity work.seg7_driver
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  beacon_id => beacon_id, new_frame => new_frame,
                  seg => seg_bus, dp => DP, an => AN);

    -- Break out the segment bus to the named XDC ports (CA..CG).
    CA <= seg_bus(0);
    CB <= seg_bus(1);
    CC <= seg_bus(2);
    CD <= seg_bus(3);
    CE <= seg_bus(4);
    CF <= seg_bus(5);
    CG <= seg_bus(6);

    -- Status LEDs (the rest are forced low).
    LED(0)            <= rx_strobe;
    LED(1)            <= new_frame;
    LED(2)            <= fifo_empty;
    LED(3)            <= fifo_full;
    LED(15 downto 4)  <= (others => '0');

    -- Passthrough: forward the raw PMOD JA1 stream out the onboard USB-UART
    -- so the host PC can capture .bin files via the same USB cable that
    -- programs the board. No re-serialization needed: both ends are 115200
    -- 8N1, so a direct wire echoes the stream byte-for-byte.
    UART_RXD_OUT <= uart_rx_pin;
end rtl;
```

Listing 1: `ingest_top.vhd` after the LH2-A passthrough edits. Two new lines: the `UART_RXD_OUT` port declaration, and the assignment of `uart_rx_pin` to that port at the bottom of the architecture.

Save (`Ctrl+S`).

Step 4.2 — Uncomment the `UART_RXD_OUT` pin in the XDC

Open the `Nexys-A7-100T-Master.xdc` you copied into the project in LH1-H Step 3. Find the line that reads:

```
#set_property -dict { PACKAGE_PIN D4    IOSTANDARD LVCMOS33 } [get_ports { UART_RXD_OUT }];
    #IO_L11N_T1_SRCC_35 Sch=uart_rxd_out
```

Remove the leading `#` so the line becomes active:

```
set_property -dict { PACKAGE_PIN D4    IOSTANDARD LVCMOS33 } [get_ports { UART_RXD_OUT }]; #
    IO_L11N_T1_SRCC_35 Sch=uart_rxd_out
```

Save the XDC. Do *not* also uncomment the `UART_TXD_IN` line directly below it — the host-to-FPGA direction is unused.

Step 4.3 — Re-synthesize and program

1. **Confirm `ingest_top` is set as the Design Sources top.** In the Sources panel's *Design Sources* tab, `ingest_top` should appear in **bold**. If it is not bold (a previous testbench or another entity may be set as top), right-click `ingest_top` → **Set as Top**. Synthesis uses the Design Sources top regardless of what is set as top in Simulation Sources; verify it explicitly before every synthesis run.

2. Flow Navigator → **Run Synthesis**. If Vivado prompts you about a stale run from LH1-H, click **Yes** to delete it. Wait for completion. Expect zero errors.

3. Flow Navigator → **Run Implementation**. Confirm WNS $\geq$ 0 ns and WHS $\geq$ 0 ns in the timing report.

4. Flow Navigator → **Generate Bitstream**.

5. Open Hardware Manager, auto-connect to the Nexys A7, and program with the new bitstream.

6. With Target A, Target B, and the Threat all powered on, confirm the seven-segment display still alternates Beacon IDs `01`/`02` as it did at the end of LH1-H. The display behavior must be *unchanged* — the passthrough is purely additive.

Step 4.4 — Identify the host-side COM port

The Nexys A7's onboard FT2232HQ exposes two USB interfaces over the same cable: one channel is the JTAG that Vivado uses to program the board, the other is a virtual COM port wired to FPGA pins `D4` (FPGA → host) and `C4` (host → FPGA). After programming, with the USB cable still connected:

1. Windows: open **Device Manager** (Win+X → Device Manager), expand **Ports (COM & LPT)**, and look for an entry such as `USB Serial Port (COM`*N*`)` that appears only when the Nexys A7 is plugged in. Note the COM number for Step 6.

2. Linux/macOS: the device appears as `/dev/ttyUSB`*N* or `/dev/tty.usbserial-`*xxx*. Run `ls /dev/tty*` before and after plugging in the board to spot the new entry.

3. If no entry appears, install the FTDI VCP driver (Windows: https://ftdichip.com/drivers/vcp-drivers/; Linux/macOS: usually built in). Re-plug the Nexys A7 after installing.

Do not unplug the board mid-capture

The same USB cable carries both JTAG and the UART channel. Do not run Vivado's Hardware Manager (or any JTAG operation) while a long capture is in progress — you will glitch the UART stream. Once the bitstream is loaded, close Hardware Manager before you start recording.

4.5 Step 5 — Save the raw-capture script

The host-side capture is a 50-line Python script. Save the listing below as `raw_capture.py` under `Code/Tools/RawCapture/`, alongside a one-line `requirements.txt` containing `pyserial>=3.5`.

```python
"""Raw byte capture from a serial port to a .bin file.

Reads bytes off a COM port and writes them verbatim to disk. Used to
produce the capture_clean.bin and capture_mixed.bin files required by
the Module 2 testbenches. The Threat's USART3 TX stream is already
present on the Nexys A7's onboard USB-UART after the LH2-A Step 4
passthrough; this script captures it from the host side.
"""
import argparse
import sys
import time
from pathlib import Path

import serial


FRAME_LEN = 43
START_DELIM = 0x7E


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("port")
    ap.add_argument("baud", type=int)
    ap.add_argument("out_bin", type=Path)
    ap.add_argument("--frames", type=int, default=2000,
                    help="stop after this many start-delimiters seen (default 2000)")
    ap.add_argument("--seconds", type=float, default=1800.0,
                    help="hard timeout in seconds (default 30 min)")
    args = ap.parse_args()

    print(f"Opening {args.port} @ {args.baud} 8N1")
    ser = serial.Serial(args.port, args.baud, timeout=1.0)

    frames = 0
    bytes_written = 0
    t_start = time.monotonic()
    deadline = t_start + args.seconds

    with args.out_bin.open("wb") as f:
        while frames < args.frames and time.monotonic() < deadline:
            chunk = ser.read(256)
            if not chunk:
                continue
            f.write(chunk)
            bytes_written += len(chunk)
            frames += chunk.count(START_DELIM.to_bytes(1, "little"))
            if frames and frames % 100 == 0:
                elapsed = time.monotonic() - t_start
                print(f"  {frames} start delimiters / "
                      f"{bytes_written} bytes / {elapsed:.1f}s",
                      end="\r")

    elapsed = time.monotonic() - t_start
    print()
    print(f"Wrote {bytes_written} bytes ({frames} start delimiters) "
          f"to {args.out_bin} in {elapsed:.1f}s")
    ser.close()


if __name__ == "__main__":
    sys.exit(main())
```

Listing 2: `Code/Tools/RawCapture/raw_capture.py` — captures raw bytes from a serial port to a `.bin` file. Stops after `--frames N` start delimiters *or* `--seconds S` elapsed, whichever comes first. Both have generous defaults so passing only the first three positional arguments is fine for most runs.

Install the dependency once:

```
cd Code/Tools/RawCapture
pip install -r requirements.txt
```

4.6 Step 6 — Record `capture_clean.bin` and `capture_mixed.bin`

Both captures must contain ≥ 1500 start delimiters. At the steady-state DATA + ACK cadence ($\sim$2 frames per second) that is roughly 12–15 minutes per recording. Run them sequentially.

Step 6.1 — Clean capture (DATA + ACK only)

1. Power on Target A, Target B, and the Threat. Confirm the Nexys A7 seven-segment display alternates `01`/`02` normally.
2. Do *not* touch the Threat's `B1` button at any point during this capture.
3. In a host terminal, from `Code/Tools/RawCapture/`, run (substitute your COM number from Step 4.4):

```
python raw_capture.py COM7 115200 ../../ingest_top/data/capture_clean.bin --frames 1500
```

4. The script prints a progress line every 100 start delimiters. Wait for it to exit on its own. Final output should report $\geq 64\,500$ bytes (1500×43, plus a few bytes of partial frame at the start).

Step 6.2 — Mixed capture (DATA + ACK with traffic gaps)

1. With the same setup still running, start a second capture:

```
python raw_capture.py COM7 115200 ../../ingest_top/data/capture_mixed.bin --frames 1500
```

2. About 30 seconds in, hold the Threat's `B1` button for $\sim$5 seconds, then release. The targeted Target goes silent during the holdoff window; the radio ping-pong stalls and resumes when the holdoff expires. On the seg7, the right-most digits will freeze on the last received Beacon ID for the duration of the hold, then resume alternating once the link recovers. The capture will contain a corresponding gap (no `0x7E` delimiters for $\sim$500 ms).
3. Optionally repeat the button hold once or twice more during the recording so the file contains multiple gaps separated by clean DATA/ACK regions. These gaps are what exercises the parser's re-sync logic in Module 2.
4. Wait for the script to exit. Final size should again be roughly $1500 \times 43 \approx 64\,500$ bytes, though wall-clock time will be slightly longer than the clean capture because of the button-hold gaps.

Why no PAUSE frames appear in the `.bin`

The Threat *transmits* PAUSE frames on the radio when its button is held; it does not *receive* its own transmissions (the SX1276 is half-duplex shadowed during its own TX). Only frames the Threat receives off-air are forwarded over `USART3` to the FPGA, so PAUSE frames never appear in the byte stream this script captures. The button hold still produces a meaningful change in the recording — the gap caused by the targeted Target going silent — and that gap is what the Module 2 testbenches use as a re-sync stress case. Module 3 introduces a separate ingest path for PAUSE traffic.

If a capture comes up short or empty

- **Script reports zero start delimiters:** wrong COM port, or the FPGA bitstream from Step 4.3 was not actually programmed (re-program and try again).
- **Script reports a few hundred start delimiters then stalls:** the Threat's `USART3 TX` DMA IRQ is not firing. Re-flash the Threat with the firmware corrected per the LH1-C Step 1.6 NVIC fix (`USART3` global interrupt enabled).
- **Bytes appear but the parse rate looks low:** confirm the seg7 is updating normally at ∼2 Hz; if not, the radio link itself is degraded.

Step 6.3 — Confirm the files landed in `data/`

1. Verify both files exist in `ingest_top/data/`:

```
ingest_top/
  data/
    capture_clean.bin     (~64.5 kB)
    capture_mixed.bin     (~64.5 kB)
  ingest_top.cache/
  ingest_top.runs/
  ingest_top.sim/
  ingest_top.srcs/
  ingest_top.xpr
```

2. LH2-G Step 3 will add a third file (`capture_corrupt.bin`), generated synthetically from `capture_mixed.bin`; you do not need to record that one now.

4.7 Step 7 — Record the LH1-H baseline utilization

1. Open the post-synthesis utilization report for the Step 4.3 run (*Reports → Report Utilization*). Record the LUT, FF, and BRAM tile counts. These are the LH1-H baseline (the passthrough adds one wire and zero logic, so the numbers are within rounding of pre-passthrough LH1-H values) — each subsequent handout adds or replaces a block and the delta will be visible in the same report format.

2. Save a screenshot of the utilization report for your deliverable.

[Figure placeholder]

`LH2A_baseline_util.png`

Post-synthesis utilization of the LH1-H baseline (after the Step 4 passthrough). Record these numbers; each Module 2 handout adds or replaces one block and the resource delta is tracked cumulatively.

Figure 1: Post-synthesis utilization of the LH1-H baseline (after the Step 4 passthrough). Record these numbers; each Module 2 handout adds or replaces one block and the resource delta is tracked cumulatively.

5 Verification Checklist

- ☐ `ingest_top.xpr` opens with all five LH1-H files present and `ingest_top` set as Top.
- ☐ `sim_1` simulation fileset exists and is empty (ready for testbenches).
- ☐ `data/` folder created in the project root.
- ☐ `ingest_top.vhd` has the `UART_RXD_OUT` port and the corresponding passthrough assignment; XDC `D4` entry is uncommented.
- ☐ New bitstream programs and the seven-segment display still alternates `01`/`02`.
- ☐ Host COM port identified and capture script runs without errors.
- ☐ `capture_clean.bin` and `capture_mixed.bin` present in `data/`, each $\geq$ 1500 start delimiters.
- ☐ LH1-H baseline utilization recorded (LUT, FF, BRAM tile counts).

Lab Handout 2-A Deliverable

1. Screenshot of the Vivado Sources panel showing all five LH1-H files and the `sim_1` fileset.
2. Updated `ingest_top.vhd` and pruned `Nexys-A7-100T-Master.xdc` (with the `UART_RXD_OUT` line uncommented).
3. `Code/Tools/RawCapture/raw_capture.py` and its `requirements.txt`.
4. Both `.bin` captures (`capture_clean.bin` and `capture_mixed.bin`) in `ingest_top/data/`, each containing ≥ 1500 start delimiters. The mixed capture must include at least one $\sim$5 s button-hold gap so that the parser's re-sync logic is exercised in Module 2.
5. Baseline post-synthesis utilization screenshot.
6. One-sentence note per LH1-H file describing what Module 2 handout replaces it (LH2-B replaces `uart_rx`; LH2-C replaces `byte_fifo`; LH2-D *deletes* `frame_detector` and adds `pkt_parser`; LH2-D updates `ingest_top` to drive the `seg7_driver` inputs from the new parser outputs — the `seg7_driver` entity itself is unchanged; LH2-F retires `seg7_driver` entirely).

Glossary

FPGA	Field-Programmable Gate Array	1, 3, 8
HDL	Hardware Description Language	2
RTL	Register-Transfer Level	1
UART	Universal Asynchronous Receiver/Transmitter	3, 5
VHDL	VHSIC Hardware Description Language	1, 2
XDC	Xilinx Design Constraints	2, 3, 4, 10